

Module 7: Git & GitHub Cheat Sheet

Setup & Configuration

Command	Description
<code>git --version</code>	Check installed Git version
<code>git config --global user.name "Name"</code>	Set your name globally
<code>git config --global user.email "email"</code>	Set your email globally
<code>git config --local user.name "Name"</code>	Set name for current repo only
<code>git config --list</code>	View all configuration settings

Creating Repositories

Command	Description
<code>git init</code>	Initialize a new Git repo in current folder
<code>git clone <url></code>	Clone (download) a remote repository

Checking Status & History

Command	Description
<code>git status</code>	Show changed/staged/untracked files
<code>git log</code>	Show full commit history
<code>git log --oneline</code>	Show compact commit history
<code>git log --graph --oneline</code>	Show history as a visual graph
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes
<code>git diff branch1..branch2</code>	Compare two branches
<code>git reflog</code>	Show all HEAD movements (safety net)

Staging Changes

Command	Description
<code>git add <file></code>	Stage a specific file
<code>git add .</code>	Stage all changes in current directory + subdirs
<code>git add --all</code> or <code>git add -A</code>	Stage ALL changes across entire project
<code>git add *</code>	Stage new/modified files (NOT deleted)
<code>git add *.txt</code>	Stage all <code>.txt</code> files in current directory
<code>git reset</code>	Unstage all changes (keep file contents)
<code>git reset <file></code>	Unstage a specific file

Committing

Command	Description
<code>git commit -m "message"</code>	Commit staged changes with a message
<code>git commit -am "message"</code>	Stage all tracked files AND commit (shortcut)
<code>git commit --amend</code>	Modify the last commit (message or content)

Undoing Changes

Command	Description
<code>git reset HEAD~</code>	Undo last commit, keep changes (unstaged)
<code>git reset --soft HEAD~</code>	Undo last commit, keep changes (staged)
<code>git reset --hard</code>	Undo last commit AND discard all changes
<code>git reset --hard <commit-id></code>	Reset to a specific commit (⚠ destructive)
<code>git checkout -- <file></code>	Discard unstaged changes in a file
<code>git restore <file></code>	Discard unstaged changes (modern syntax)
<code>git restore --staged <file></code>	Unstage a file (modern syntax)

Deleting Files

Command	Description
<code>git rm <file></code>	Delete file + stage the deletion
<code>git rm -f <file></code>	Force delete (even with local changes)
<code>git rm --cached <file></code>	Untrack file but keep it on disk
<code>git rm -r <folder></code>	Recursively delete folder + contents

Branching

Command	Description
<code>git branch</code>	List all local branches
<code>git branch -a</code>	List all branches (local + remote)
<code>git branch <name></code>	Create a new branch
<code>git checkout <name></code>	Switch to a branch
<code>git switch <name></code>	Switch to a branch (modern syntax)
<code>git checkout -b <name></code>	Create + switch to a new branch
<code>git switch -c <name></code>	Create + switch (modern syntax)
<code>git branch -d <name></code>	Delete a merged branch
<code>git branch -D <name></code>	Force delete any branch
<code>git branch -m <new-name></code>	Rename current branch

Merging & Rebasing

Command	Description
<code>git merge <branch></code>	Merge a branch into current branch
<code>git merge --abort</code>	Cancel a merge in progress
<code>git rebase <branch></code>	Rebase current branch onto another
<code>git rebase -i HEAD~n</code>	Interactive rebase (edit last n commits)
<code>git rebase --abort</code>	Cancel a rebase in progress

Stashing

Command	Description
<code>git stash</code>	Save uncommitted changes temporarily
<code>git stash save "message"</code>	Stash with a descriptive name
<code>git stash pop</code>	Restore most recent stash + remove it
<code>git stash apply</code>	Restore most recent stash + keep it
<code>git stash list</code>	Show all saved stashes
<code>git stash apply stash@{n}</code>	Restore a specific stash
<code>git stash drop stash@{n}</code>	Delete a specific stash
<code>git stash clear</code>	Delete ALL stashes

Remote Repositories

Command	Description
<code>git remote -v</code>	View remote connections
<code>git remote add origin <url></code>	Add a remote named "origin"
<code>git remote set-url origin <url></code>	Change remote URL
<code>git remote remove <name></code>	Remove a remote
<code>git push -u origin main</code>	Push + set upstream (first time)
<code>git push</code>	Push to default upstream
<code>git push origin <branch></code>	Push a specific branch
<code>git push --force</code>	Force push (⚠️ overwrites remote)
<code>git pull</code>	Fetch + merge from remote
<code>git pull origin <branch></code>	Pull a specific branch
<code>git fetch</code>	Download changes without merging
<code>git fetch origin</code>	Fetch from a specific remote

Tags

Command	Description
<code>git tag</code>	List all tags
<code>git tag <name></code>	Create a lightweight tag
<code>git tag -a <name> -m "msg"</code>	Create an annotated tag
<code>git push origin <tag></code>	Push a specific tag
<code>git push origin --tags</code>	Push all tags
<code>git tag -d <name></code>	Delete a local tag

.gitignore Patterns

Pattern	Meaning
<code>filename</code>	Ignore a specific file
<code>folder/</code>	Ignore an entire folder
<code>*.ext</code>	Ignore all files with extension
<code>!keep.ext</code>	Exception — do NOT ignore this file
<code>**/logs</code>	Ignore <code>logs</code> anywhere in project
<code>doc/*.txt</code>	Ignore <code>.txt</code> only in <code>doc/</code> folder

Common Workflows

Daily Workflow:

```
git status          # Check what changed
git add .           # Stage everything
git commit -m "msg" # Save permanently
git push            # Upload to GitHub
```

Starting a New Feature:

```
git checkout -b feature/name # Create feature branch
# ... do work ...
git add .
git commit -m "Add feature"
git push origin feature/name
# Create Pull Request on GitHub
```

Updating Your Branch:

```
git checkout main
git pull
git checkout feature/name
git merge main # or: git rebase main
```

Resolving Merge Conflicts:

```
git merge feature-branch # Conflict occurs
# Edit conflicted files, remove markers
git add .
git commit -m "Resolve conflicts"
```
